# Programação Orientada a Objetos

## Aula 06 - Hierarquia de Classes

Prof. Dr. Eduardo de Olivindo (eduardo.olivindo@ifce.edu.br)

Tecnologia em Análise e Desenvolvimento de Sistemas

IFCE - Campus Itapipoca



# Objetivos da aula

- Compreender herança como mecanismo de especialização entre classes.
- Identificar superclasses, subclasses e a relação de "é um".
- Reutilizar código com herança e `super()`.
- Sobrescrever métodos com segurança e reconhecer os limites da herança.
- Compreender o polimorfismo como um dos pilares da POO e reconhecer sua aplicação.

# Herança entre Classes

# O que é herança?

- Herança permite criar uma nova classe a partir de uma classe existente.
- A classe nova recebe atributos e métodos da classe original.
- A subclasse pode:
  - adicionar comportamentos;
  - especializar comportamentos existentes;
  - reutilizar a implementação da superclasse.
- Pra que isso???
  - Reutilização de código, organização hierárquica, extensibilidade e polimorfismo
- A relação modelada é de **"é um"**, e não de "tem um".

# Herança: exemplo do domínio

- `Cachorro` **é um** `Animal`.
- `Gato` **é um** `Animal`.
- Todo animal pode ter um nome e emitir um som.
- Cada espécie pode emitir um som diferente.



# Herança vs associação

- **Herança:** relação de "é um".
  - `Cachorro` é um `Animal`.
- **Associação:** relação de "tem um" ou "usa um".
  - `Carro` tem um `Motor`.
- Uma classe pode herdar de outra e também possuir objetos como atributos.
- Escolha a relação que representa melhor o domínio.

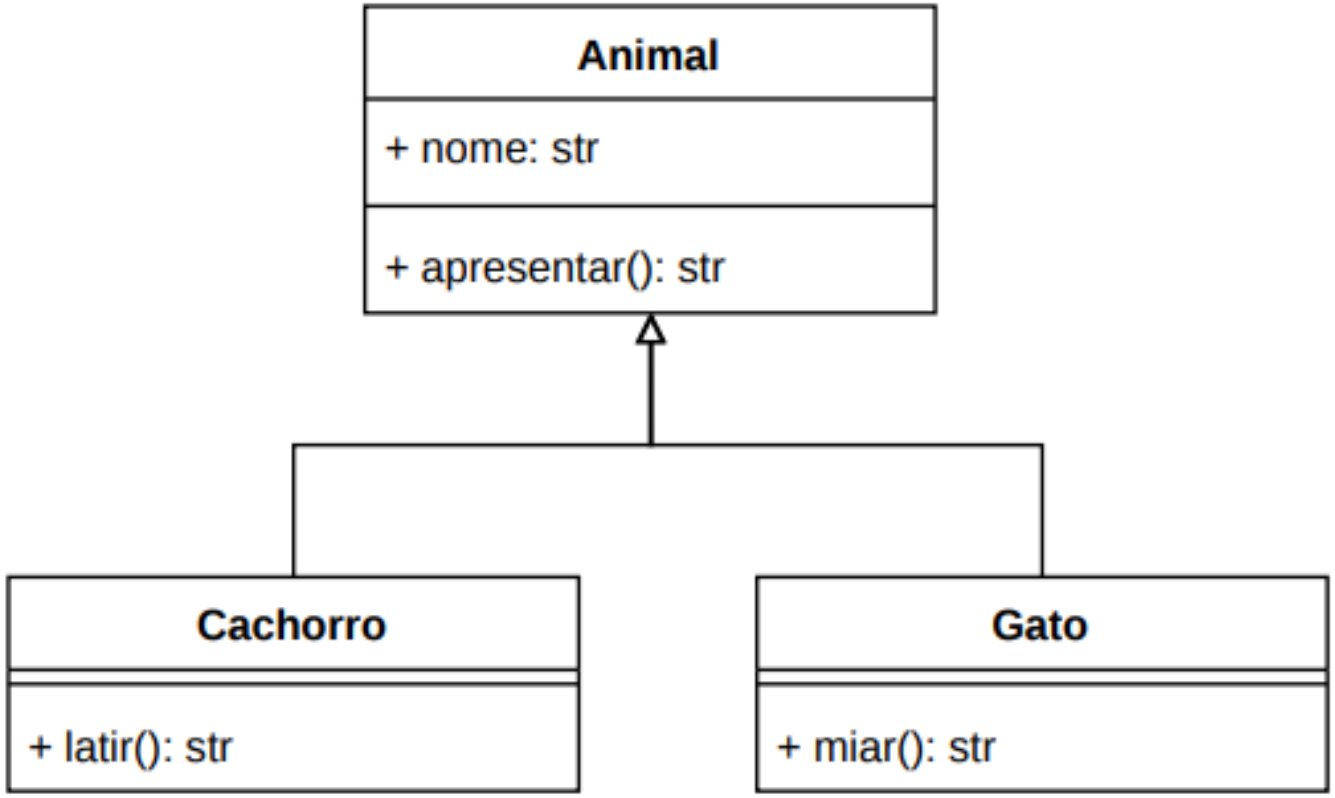
# Exercício 1: herança ou associação?

- `Professor` e `Pessoa`
- `Pedido` e `Produto`
- `Cachorro` e `Animal`
- `Carro` e `Motor`
- `ContaCorrente` e `Conta`

# Exercício 1: Respostas

- `Professor` é uma `Pessoa`: **herança**.
- `Pedido` tem um `Produto`: **associação**.
- `Cachorro` é um `Animal`: **herança**.
- `Carro` tem um `Motor`: **associação**.
- `ContaCorrente` é uma `Conta`: **herança**.

# Herança: diagrama de classes



# Herança: sintaxe básica

```
class Animal:       # SUPERCLASSE OU CLASSE PAI
    def __init__(self, nome):
        self.nome = nome

    def apresentar(self):
        return f"Eu sou {self.nome}."

class Cachorro(Animal):  # SUBCLASSE OU CLASSE FILHA
    def latir(self):
        return "Au au!"
```

- `Cachorro(Animal)` indica que `Cachorro` herda de `Animal`.
- Um objeto `Cachorro` possui `nome` e `apresentar()`.
- A subclasse também pode declarar novos métodos.

# Herança: testando hierarquia

```
rex = Cachorro("Rex")
print(rex.nome)
print(rex.apresentar())
print(rex.latir())
```

- O objeto `rex` foi criado pela classe `Cachorro`.
- Mesmo assim, ele usa o atributo e o método definidos em `Animal`.
- Esse é o reaproveitamento oferecido pela herança.

## Exercício 2: Crie e teste a subclasse Gato!

# Vamos Fazer Juntos:

### Modele uma hierarquia

- Crie uma classe `Veiculo` com os atributos `marca` e `modelo`.
- Adicione um método `descricao()` que retorne esses dados.
- Crie a subclasse `Carro`, com o atributo extra `quantidade_portas`.
- Instancie um carro e chame os métodos herdados.

# Uma solução possível

- É possível **sobrescrever** o construtor da superclasse:

```
class Veiculo:
    def __init__(self, marca, modelo):
        self.marca = marca
        self.modelo = modelo

    def descricao(self):
        return f"{self.marca} {self.modelo}"

class Carro(Veiculo):
    def __init__(self, marca, modelo, quantidade_portas): # SOBRESCRITA
        self.marca = marca
        self.modelo = modelo
        self.quantidade_portas = quantidade_portas
```

# Outra solução possível

```
class Veiculo:
    def __init__(self, marca, modelo):
        self.marca = marca
        self.modelo = modelo

    def descricao(self):
        return f"{self.marca} {self.modelo}"

class Carro(Veiculo):
    def __init__(self, marca, modelo, quantidade_portas):
        super().__init__(marca, modelo) # CHAMANDO O MÉTODO DA SUPERCLASSE
        self.quantidade_portas = quantidade_portas
```

# `super()` e inicialização

- `super()` permite acessar métodos da superclasse, especialmente quando a subclasse deseja reutilizar sua implementação.
  - `super().__init__(...)` chama o construtor da classe pai.
- Assim, a subclasse não precisa duplicar a inicialização de `marca` e `modelo`.
- Use `super()` quando a subclasse acrescentar dados ao que já existe.

```
meu_carro = Carro("Fiat", "Argo", 4)
print(meu_carro.descricao())
print(meu_carro.quantidade_portas)
```

- `super()` pode ser usado pra acessar qualquer método da superclasse.

# Exercício 3: especialize uma classe

- **Crie uma classe** `Pessoa` com os atributos `nome` e `idade`.
  - Adicione um método `apresentar_detalhes()` que retorne:
    - `Nome: Ana - idade: 20`
- **Crie a subclasse** `Aluno`, acrescentando o atributo `matricula`.
  - Use `super().__init__()` para inicializar `nome` e `idade`.
  - Crie `apresentar_detalhes()`, reutilizando o método da superclasse.
- Teste com:

```
aluno = Aluno("Ana", 20, "2026001")
print(aluno.apresentar_detalhes()) # Nome: Ana - idade: 20 - Matrícula: 2026001
```

# Mais sobre sobrescrita de métodos

- A sobrescrita possibilita que objetos de diferentes classes respondam ao mesmo método de maneiras distintas, permitindo tratá-los de forma uniforme através de uma interface comum.

```
class Animal:
    def __init__(self,nome): self.nome=nome
    def emitir_som(self): return "Som genérico"

class Gato(Animal):
    def emitir_som(self): # INTERFACE COMUM
        return "Miau!"

class Cachorro(Animal):
    def emitir_som(self):# INTERFACE COMUM
        return "Au Au!"
```

# Polimorfismo

- A capacidade de tratar objetos de diferentes classes de forma uniforme através de uma interface comum é chamada de **Polimorfismo**.

```
def todomundofazbarulho(animais: list[Animal]):
    for animal in animais:
        print(animal.nome, animal.emitir_som())

animais = [Cachorro("Rex"), Gato("Mimi")]
todomundofazbarulho(animais)
```

- O código percorre objetos de tipos diferentes.
- Todos respondem à mesma mensagem: `emitir_som()`.
- Cada classe decide qual comportamento executar.

# Exercício 4: Funcionários

- Crie uma classe `Funcionario` com `nome` e método `calcular_pagamento()` que retorna um salário fixo (salário mínimo).
- Crie `FuncionarioHorista`, que recebe `horas` e `valor_hora`.
- Sobrescreva `calcular_pagamento()` para retornar `horas * valor_hora`.
- Crie `FuncionarioComissionado` que recebe o salário fixo mais um valor de comissão.
- Crie uma função que receba uma lista de funcionários e calcule e imprima o salário de todos eles.

# Exercício 5: Hierarquia de Pessoas

**Modele a hierarquia:**

- Toda **Pessoa** tem nome e cpf.
- Todo **Servidor** é uma pessoa e possui um código siape.
- Todo **Professor é um Servidor** e possui uma lista de disciplinas.
- Todo **TAE é um Servidor** e possui uma função.
- Todo **Aluno é uma Pessoa** e possui uma matricula, um curso e uma lista de disciplinas.
- Uma **Disciplina** possui um nome e uma CH.
- Todas as Pessoas se apresentam conforme sua subclasse.

# Quando evitar herança?

- Não use herança apenas para reaproveitar algumas linhas de código.
- Se a relação não for realmente de "é um", considere composição ou associação.
- Hierarquias muito profundas ficam difíceis de entender e alterar.
- Prefira classes pequenas, responsabilidades claras e contratos simples.

# Fechamento

- Herança cria uma relação entre uma classe mais geral e outra mais específica.
- `super()` permite reutilizar a implementação da superclasse.
- Sobrescrita especializa comportamentos.
- Polimorfismo permite tratar objetos diferentes por uma mesma interface.
- A melhor hierarquia é a que torna o modelo mais claro, e não apenas mais curto.